Ministry of Education, Culture and Research of the Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatic

ANDREI ELCISCEV
Group FAF-243

LABORATORY WORK No. 13
SEMAPHORES IN YOUR OWN KERNEL

Checked by:
Reiţman Patricia

Chişinău, 2026

Contents

1 Introduction **2**

2 Part 1: Numbers, Stubs, and User Programs **2**
2.1 Checkpoint Before Kernel Dispatch 3

3 Part 2: Kernel Implementation **3**
3.1 Checkpoint With Kernel Dispatch 4

4 Part 3: A Semaphore as a Lock **4**

5 Compatibility and Scope **5**

6 Conclusion **6**

A New xv6 Source Files **6**

1 Introduction

This laboratory work implements counting semaphores in xv6. Three new system calls initialize a semaphore, acquire a permit, or release a permit. Eight slots, numbered 0 through 7, are shared by all processes. Each slot has a count and a spinlock. Waiting processes sleep on the address of the slot and can be woken by a release.

The work took place in `/home/user/os-lab13/xv6-riscv`, a separate copy of the Lab 10 xv6 tree in the Linux VM. The copied baseline built and booted in QEMU before any edits. All displayed console output comes from the recorded runs. Compiler invocations that succeeded are summarized in the accompanying transcript.

2 Part 1: Numbers, Stubs, and User Programs

`SYS_sync` remains 22 and `SYS_trace` remains 23. The three semaphore calls use 24, 25, and 26. The numbers were added to `kernel/syscall.h`, the declarations to `user/user.h`, and the stub entries to `user/usys.pl`. The Makefile adds `semtest` and `mix` to the programs included in the filesystem image.

```
Baseline source: /home/user/os-lab10/xv6-riscv
$ uname -sr
Linux 6.14.0-37-generic
exit code: 0
$ riscv64-linux-gnu-gcc --version
riscv64-linux-gnu-gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

exit code: 0
$ qemu-system-riscv64 --version
QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.18)
Copyright (c) 2003-2023 Fabrice Bellard and the QEMU Project developers
exit code: 0
$ echo baseline boot OK
baseline boot OK
```

Listing 1: Tool versions and baseline QEMU shell check

```
$ sed -n /^sem_init:/,$p user/usys.S
sem_init:
 li a7, SYS_sem_init
 ecall
 ret
.global sem_down
sem_down:
 li a7, SYS_sem_down
 ecall
 ret
.global sem_up
sem_up:
 li a7, SYS_sem_up
 ecall
 ret
exit code: 0
```

Listing 2: Generated RISC-V stubs load the call number in a7 and execute ecall

2.1 Checkpoint Before Kernel Dispatch

With the user-side changes in place, the kernel received the new call numbers and reported them as unknown. The child printed before the parent because its `sem_down` returned an error instead of sleeping. The user program's final OK message at this checkpoint is only a print statement and does not establish correct synchronization.

```
$ semtest
3 semtest: unknown sys call 24
4 semtest: unknown sys call 25
child: I ran after the parent
parent: done, I let the child go
3 semtest: unknown sys call 26
3 semtest: unknown sys call 24
3 semtest: unknown sys call 24
3 semtest: unknown sys call 25
5 semtest: unknown sys call 25
  pipong 1
5 semtest: unknown sys call 26
5n semtest: unknown sys call 25
g 1
3 semtest: unknown sys call 26
3 semtest: unknown sys call 25
ping 2
3 semtest: unknown sys call 26
3 semtest: unknown sys call 25
  pong 2
5 semtest: unknown sys call 26
5 semtest: unknown sys call 25
  pong ping 33
3 semtest: unknown sys call 26

5 semtest: unknown sys call 26
semtest: OK
```

Listing 3: Actual QEMU transcript before installing the handlers

3 Part 2: Kernel Implementation

`kernel/sem.c` defines the shared table and validates IDs. Initialization rejects negative values. The Makefile adds `sem.o` to the kernel build, and `kernel/syscall.c` maps each new number to its handler. Since the base already supported `trace`, its syscall-name table was extended too.

The acquisition path takes the slot spinlock and checks the count. With no permit available, it calls `sleep_prepare(s)` while holding the lock, releases the lock, and calls `sleep()`. A release between registration and the actual sleep clears the registered channel, so the sleeper does not miss that semaphore wakeup. The acquire path then retakes the

lock and checks the count again before decrementing it. The release path increments the count and calls `wakeup(s)` on the same address.

3.1 Checkpoint With Kernel Dispatch

The parent waited ten ticks before releasing the child. The child printed after the parent. Two further semaphore slots alternated the turn between ping and pong, yielding three ordered pairs and `semtest: OK`.

```
$ semtest
parent: done, I let the child go
child: I ran after the parent
ping 1
  pong 1
ping 2
  pong 2
ping 3
  pong 3
semtest: OK
```

Listing 4: Actual QEMU transcript after installing the handlers

4 Part 3: A Semaphore as a Lock

QEMU ran with three virtual CPUs. The two unlocked runs showed mixed letters. With the lock enabled, all ten lines remained intact. The verification counted 150 letters from each process and checked that the locked run contained five 30-character lines of each type.

```
$ mix
AAAAAAAAAAbAbAbbbbbbbbbbbAAbAbAAAAAAAAbbAbAAAAAbAb
AbAAAAbAAAAAAAbAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAbAbbbbbbbb
bbbbbbbbbbbbbbbbbbbAbAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
bbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
$ mix
AAAAbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bAAAAAAAAAAAAAAAAbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbAAAAAbAbbbAAAAb
bbbbbbbbAAAAAAAAAAAAAAAAAAAAAAbAAAAAbbbAA
AAAAAbbbbbbAAAAAAbbAAAAAAAAAAAAAAAAAA
AAAAAAAA
AAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
$ mix lock
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
```

Listing 5: Two unlocked mix runs and one locked run

Observe 1. Why do letters mix without the lock? Each letter is a separate `write(1, p, 1)` call. A line costs 31 write system calls: 30 letters and one newline. The two processes can execute their writes in between one another, so neither owns the console for the duration of a line.

Observe 2. Which lines enclose the critical section? `sem_down(3);` before the loop that writes the line, and `sem_up(3);` after it. Slot 3 starts at one, so a second process cannot start a line before the current owner returns the permit.

Observe 3. Why does sem_down recheck with while? `sem_up` wakes all processes registered on the slot. When one wakes, another process may acquire the lock first and consume the available permit. The woken process must recheck the value under the lock and wait again if it is zero.

Observe 4. Why release the spinlock before sleep? `sem_up` must acquire the same spinlock to add a permit and wake the waiter. Sleeping with it held blocks the only operation that can make progress, and xv6's scheduler also rejects sleeping with the extra spinlock held. Registering the wait channel before release prevents a lost semaphore wakeup if the post happens before the actual sleep.

5 Compatibility and Scope

Mask 117440512 enables bits 24, 25, and 26. The existing Lab 10 tracer reported all three new syscall names while `semtest` still completed.

```
$ trace 117440512 semtest
3: syscall sem_init -> 0
parent: done, I let the child go
3: syscall sem_up -> 0
4: syscall sem_down -> 0
child: I ran after the parent
3: syscall sem_init -> 0
3: syscall sem_init -> 0
3: syscall sem_down -> 0
ping 1
3: syscall sem_up -> 0
5: syscall sem_down -> 0
  pong 1
5: syscall sem_up -> 0
3: syscall sem_down -> 0
ping 2
```

```
3: syscall sem_up -> 0
5: syscall sem_down -> 0
  pong 2
5: syscall sem_up -> 0
3: syscall sem_down -> 0
ping 3
3: syscall sem_up -> 0
5: syscall sem_down -> 0
  pong 3
5: syscall sem_up -> 0
semtest: OK
```

Listing 6: The new calls selected by the Lab 10 trace mask

The handout's API uses shared numbered slots and assumes that initialization is done before concurrent use. Its killed check runs after sleep returns. The baseline can miss a kill that arrives between wait registration and the transition to SLEEPING, because kkill only wakes processes that are already asleep. The lab tests exercise semaphore wakeups and do not cover that cancellation race.

6 Conclusion

Registering the wait before releasing the spinlock lets the kernel coordinate sleep with permit release. The count and the loop around the wait let several processes compete safely for permits. The tests used a zero-count slot for signaling, two slots for alternating turns, and a one-permit slot to protect complete output lines.

A New xv6 Source Files

```
// sem.c: counting semaphores inside the xv6 kernel (Lab 13).
// A small table of NSEM semaphores, shared by every process.
// Each one is a number plus a spinlock that protects the number.

#include "types.h"
#include "riscv.h"
#include "defs.h"
#include "param.h"
#include "spinlock.h"
#include "proc.h"

#define NSEM 8

struct sem {
  struct spinlock lock;   // a zeroed spinlock is a free spinlock
  int value;              // how many may still pass
};

static struct sem sems[NSEM];

// sem_init(id, value): give semaphore id its start value.
uint64
sys_sem_init(void)
{
```

```
  int id, value;
  argint(0, &id);
  argint(1, &value);
  if (id < 0 || id >= NSEM || value < 0)
    return -1;
  acquire(&sems[id].lock);
  sems[id].value = value;
  release(&sems[id].lock);
  return 0;
}

// sem_down(id): take one unit; if there is none, sleep until there is.
uint64
sys_sem_down(void)
{
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];

  acquire(&s->lock);
  while (s->value == 0) {
    sleep_prepare(s);       // I will sleep on the address of s
    release(&s->lock);      // never sleep holding a spinlock
    sleep();                // returns at once if sem_up came in between
    acquire(&s->lock);
    if (killed(myproc())) { // kill must still work on a waiting process
      release(&s->lock);
      return -1;
    }
  }
  s->value--;
  release(&s->lock);
  return 0;
}

// sem_up(id): give one unit back and wake whoever sleeps on s.
uint64
sys_sem_up(void)
{
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];

  acquire(&s->lock);
  s->value++;
  wakeup(s);
  release(&s->lock);
  return 0;
}
```

Listing 7: kernel/sem.c

```
// semtest.c: two tests for the kernel semaphores (Lab 13).
```

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(void)
{
  // Test 1: order. Semaphore 0 starts at 0, so the child must wait.
  sem_init(0, 0);
  if (fork() == 0) {
    sem_down(0);
    printf("child: I ran after the parent\n");
    exit(0);
  }
  pause(10);
  printf("parent: done, I let the child go\n");
  sem_up(0);
  wait(0);

  // Test 2: ping-pong. Semaphore 1 = ping's turn, semaphore 2 = pong's turn.
  sem_init(1, 1);
  sem_init(2, 0);
  if (fork() == 0) {
    for (int i = 1; i <= 3; i++) {
      sem_down(2);
      printf("  pong %d\n", i);
      sem_up(1);
    }
    exit(0);
  }
  for (int i = 1; i <= 3; i++) {
    sem_down(1);
    printf("ping %d\n", i);
    sem_up(2);
  }
  wait(0);
  printf("semtest: OK\n");
  exit(0);
}
```

Listing 8: user/semtest.c

```c
// mix.c: two processes print five lines each, one letter per write().
//   mix        no lock: the letters of the two processes can mix
//   mix lock   semaphore 3 used as a lock: every line stays whole
#include "kernel/types.h"
#include "user/user.h"

void
lines(char *s, int uselock)
{
  for (int k = 0; k < 5; k++) {
    if (uselock)
      sem_down(3);
    for (char *p = s; *p; p++)
      write(1, p, 1);
    if (uselock)
      sem_up(3);
```

```
  }
}

int
main(int argc, char *argv[])
{
  int uselock = (argc > 1);
  sem_init(3, 1);
  if (fork() == 0) {
    lines("bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb\n", uselock);
    exit(0);
  }
  lines("AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\n", uselock);
  wait(0);
  exit(0);
}
```

Listing 9: user/mix.c